

Architectural Design

Architecture Decision Records (ADRs) · Decisions on authentication, calendar functionality and notification delivery, along with architectural patterns and UML modeling.

3.1 Architectural Pattern

The UniMentor platform adopts a **Layered Architecture** pattern (also known as N-Tier architecture) for its backend, separating concerns into distinct, loosely coupled layers. This approach is highly suitable for our stack (Node.js, Express, PostgreSQL) and directly addresses our maintainability and scalability goals as outlined in the requirements.

Decoupling the UI from the Database

A primary benefit of this pattern is that the User Interface (the HTML/CSS/JS frontend hosted on Vercel) never interacts directly with the database. The decoupling happens through a strict hierarchy where each layer only communicates with the layer immediately below it.

1. Presentation Layer (Frontend / UI):

- **Responsibility:** The client-side application. It handles user interactions, renders the DOM, and communicates with the backend exclusively via RESTful HTTP calls (fetching data, submitting forms).
- **Technology:** Vanilla HTML, CSS, and JavaScript.
- **Isolation:** It is completely unaware of PostgreSQL, database schemas, or how data is persisted. It only expects JSON responses from specific API endpoints.

2. API / Routing Layer (Controllers):

- **Responsibility:** Located in the Node.js backend. This layer receives the HTTP requests from the UI, validates the basic payload (e.g., checking if an email string is valid), and maps the request to the correct business operation.
- **Technology:** Express.js Routers and Controllers.
- **Isolation:** It handles HTTP status codes (200, 400, 401) and formatting the JSON response, but it contains zero business logic and zero database queries.

3. Business Logic Layer (Services):

- **Responsibility:** This layer contains the core rules of UniMentor (e.g., checking if a slot is 2 hours away, ensuring a student doesn't exceed 3 active bookings, verifying

no overlap).

- **Technology:** Plain JavaScript functions/classes in Node.js.
- **Isolation:** It processes the requests without knowing anything about HTTP statuses or raw database queries. It relies on the Data Access layer to fetch data, applies the rules, and returns the result to the Controller.

4. Data Access Layer (Repositories):

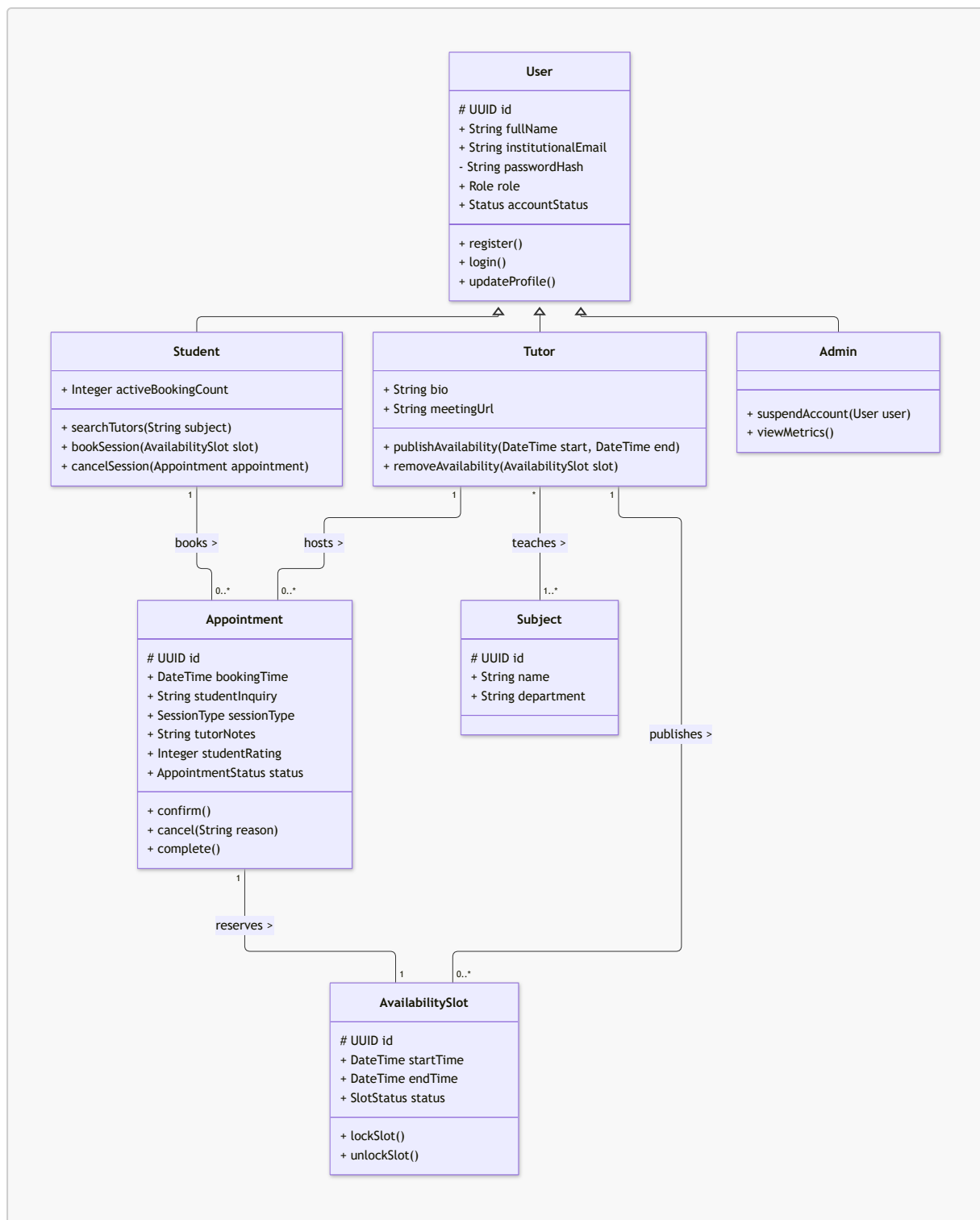
- **Responsibility:** This layer handles all direct communication with the PostgreSQL database. It manages transactions, executes queries, and maps database rows to JavaScript objects.
- **Technology:** `pg` (node-postgres) library, parameterized SQL queries.
- **Isolation:** It prevents SQL injection and encapsulates all schema knowledge. The layers above do not know table names or column structures.

Why this decoupling matters: By using this structure, if we were to swap our vanilla JS frontend for a React application, or migrate from PostgreSQL to another relational database, the changes would be isolated to a single layer without breaking the entire system.

3.2 UML Modeling

3.2.1 Class Diagrams

The following class diagram represents the core domain model of UniMentor, focusing on users, tutoring subjects, availability, and appointments. It includes multiplicity (1:N between Tutor and AvailabilitySlot, M:N between Tutor and Subject) and standard visibility modifiers (+ public, - private, # protected).



The Tutor–Subject relationship is many-to-many (a tutor teaches one or more subjects and a subject is taught by many tutors); in the database it is implemented with the `tutor_subjects` join table. `SessionType` is an enumeration (QUICK_CHECK, STANDARD, DEEP_DIVE) chosen at booking.

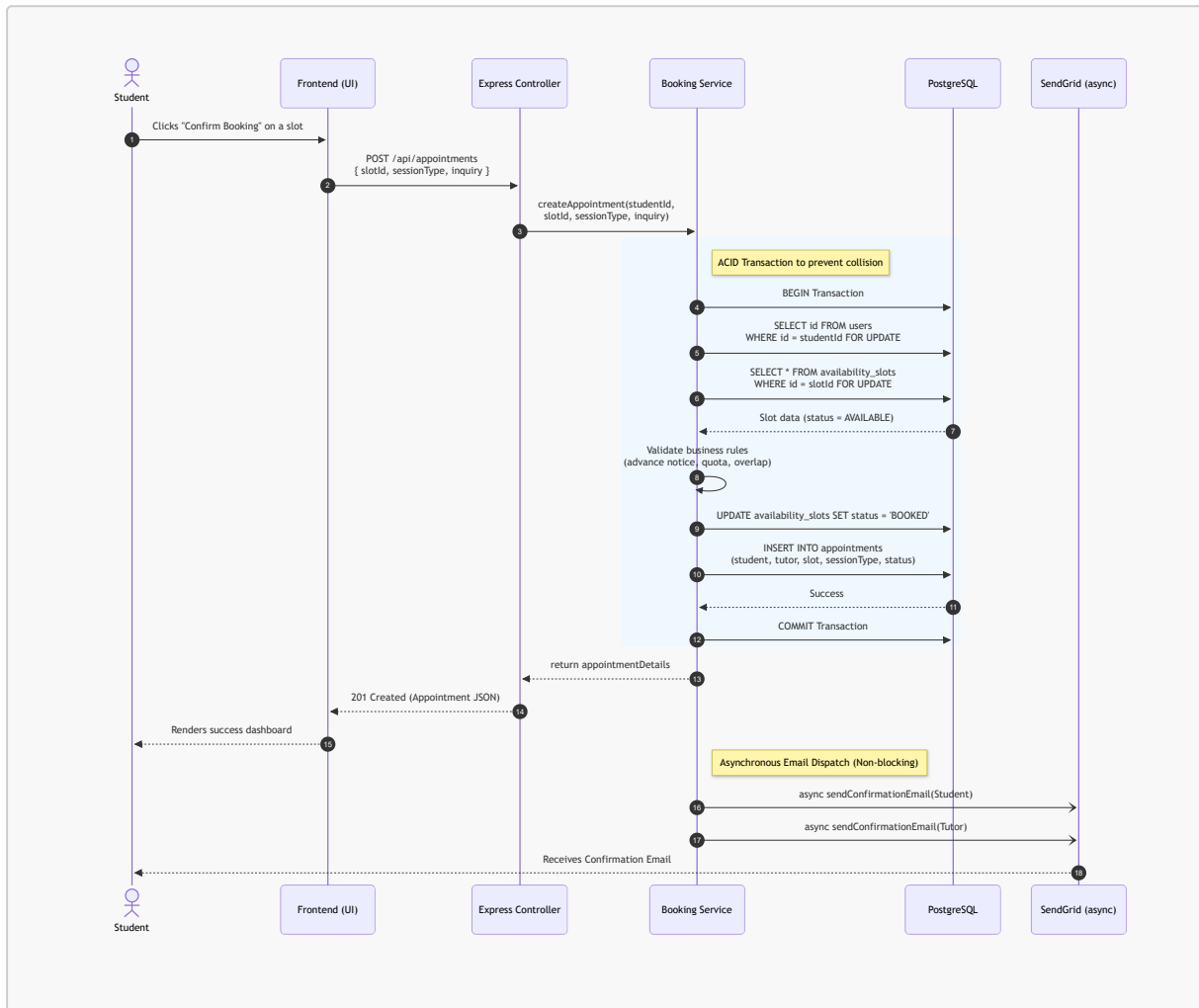
3.2.2 Sequence Diagrams

To fully illustrate the behavior of the layered architecture and the asynchronous operations required by our Non-Functional Requirements, the following sequence diagrams map our most

critical use cases.

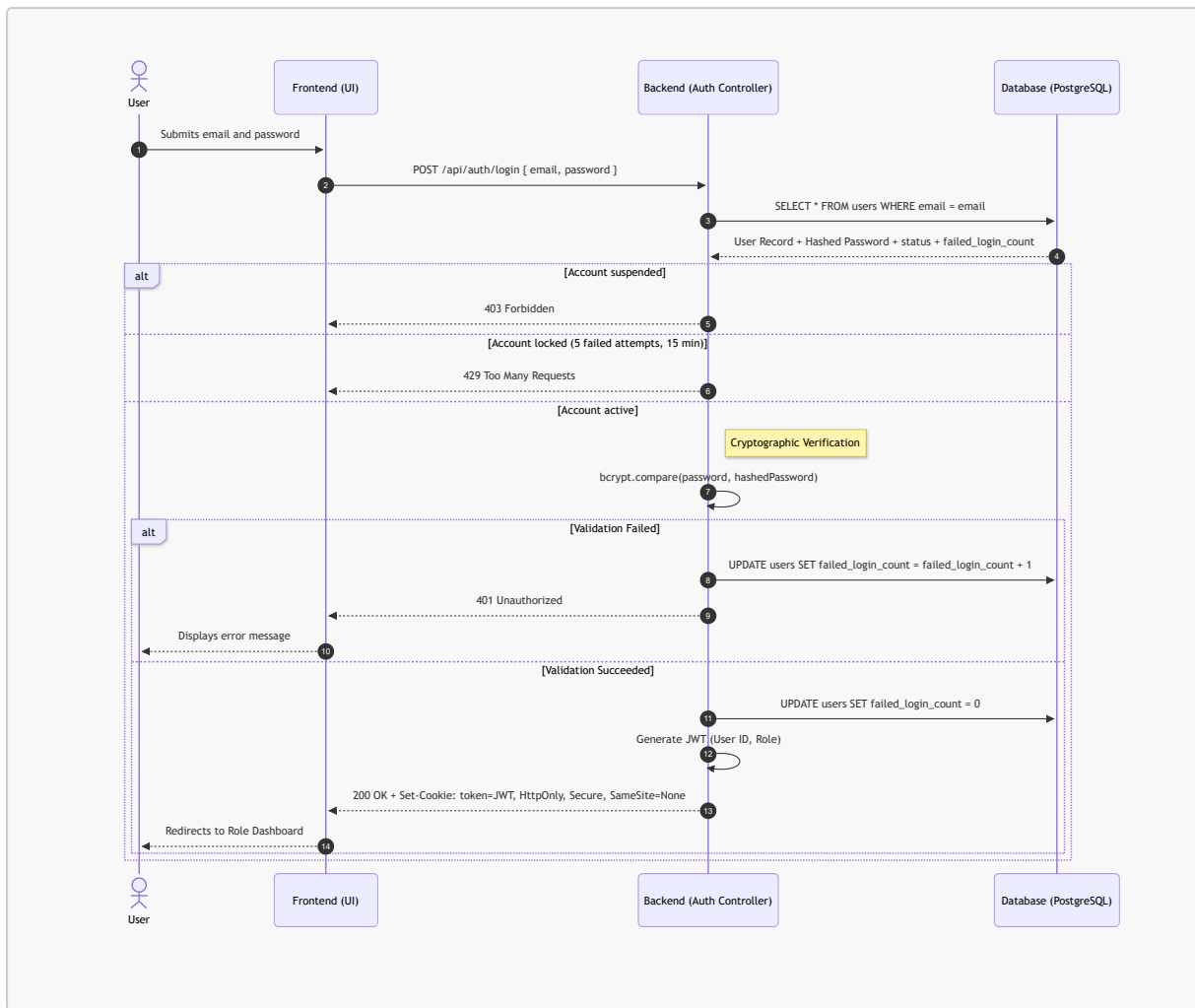
A. Booking Transaction (CU-08)

This diagram illustrates the core booking flow, explicitly highlighting the database transaction isolation (to prevent double-booking) and the **asynchronous calls** used for the notification service (SendGrid). The lock on the student row serializes simultaneous bookings by the same student, so the 3-booking quota and the student-overlap rule cannot be bypassed by parallel requests.



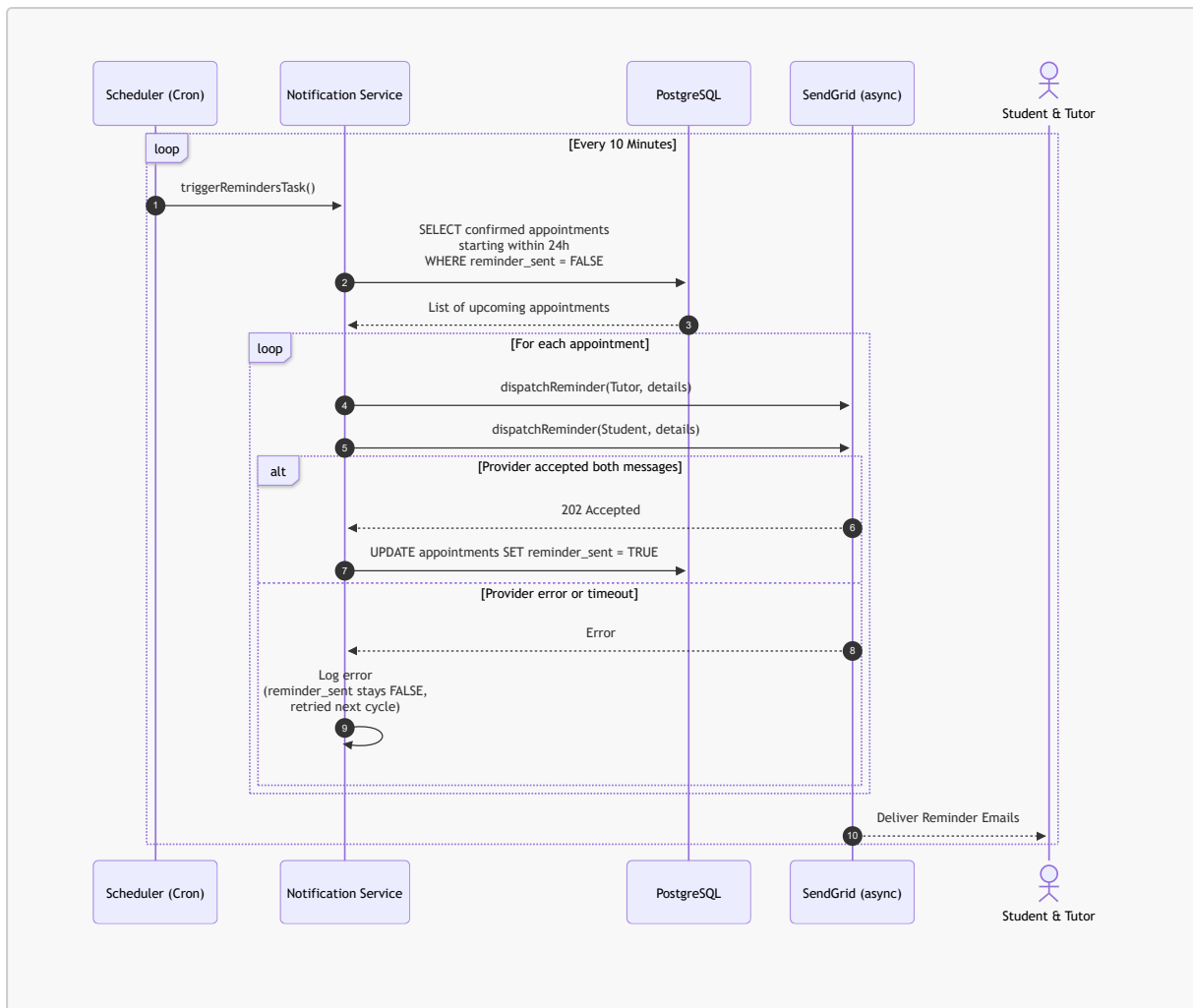
B. User Authentication (CU-02)

This diagram illustrates the login process and how the backend manages secure session tokens (HttpOnly Cookies) without exposing them to the frontend scripts.



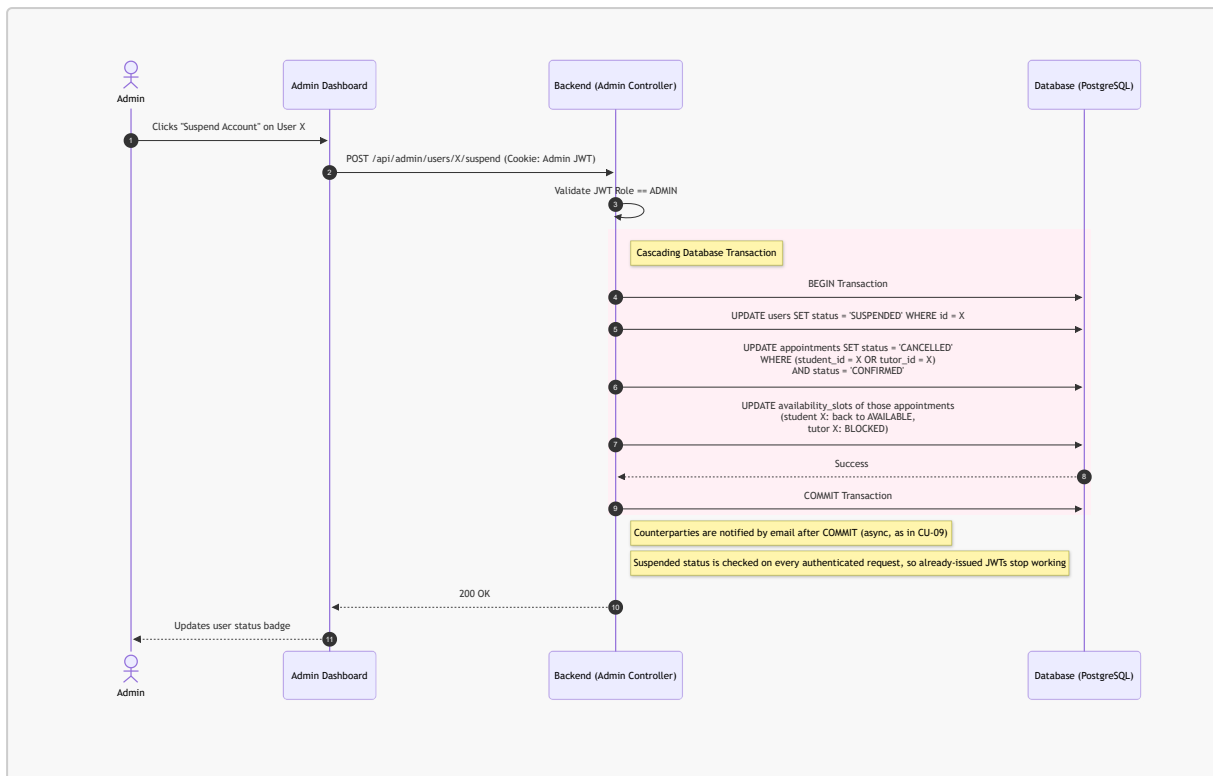
C. Automated Reminders Dispatch (CU-10)

This diagram demonstrates a background cron job, which runs outside the request/response cycle (asynchronous by nature), processing upcoming appointments and dispatching reminders. The `reminder_sent` flag is updated only after the provider accepts the messages, so failed sends are retried in the next cycle.



D. Account Suspension (CU-13)

This diagram maps an administrative action modifying user state and clearing future dependencies (canceling upcoming bookings and releasing their slots, so a suspended student's slots become bookable again).



3.3 Decision Justification (ADRs)

Below are the final **Architecture Decision Records (ADRs)** for the UniMentor project, documenting the technical choices made during the planning phase.

ADR 1: Backend Framework Selection

- **Context:** The backend needs to handle REST API requests, interact with a relational database, and manage asynchronous tasks like email dispatching. The team has diverse language experiences.
- **Options considered:**
 1. *Python (Django/FastAPI):* Excellent for rapid development and data processing, but introduces a context switch from the JavaScript-heavy frontend.
 2. *Java (Spring Boot):* Highly robust for enterprise applications, but overly verbose and complex for a rapid pilot project.
 3. *Node.js with Express:* Lightweight, highly performant for I/O operations (like database queries and API calls), and allows the team to use JavaScript across the entire stack.

Outcome: We chose Node.js with Express. It unifies our language ecosystem, simplifies context switching for developers, and provides an immense ecosystem of libraries (npm) perfectly suited for building RESTful APIs rapidly.

ADR 2: Primary Database Technology

- **Context:** The system must manage users, subjects, availability slots, and appointments. The most critical risk identified in the requirements is preventing schedule conflicts (double-bookings).
- **Options considered:**
 1. *MongoDB (NoSQL):* Flexible schema and fast prototyping. However, handling complex relational joins (e.g., finding a tutor's available slots for a specific subject) and ensuring strict atomic transactions is more difficult.
 2. *PostgreSQL (Relational SQL):* A highly mature relational database. It provides strict ACID compliance, relational integrity (foreign keys), and built-in transaction locking mechanisms.

Outcome: We chose PostgreSQL. The relational nature of our data (Users -> Appointments <- Slots) is a perfect fit. More importantly, PostgreSQL's row-level locking (**SELECT FOR UPDATE**) natively solves the race condition risk (double-booking) required by our non-functional constraints.

ADR 3: Frontend Technology Stack

- **Context:** The user interface needs to be responsive, accessible, and communicate with our REST API. We are building a pilot prototype with a tight deadline.
- **Options considered:**
 1. *React / Next.js:* Industry standard for complex SPAs (Single Page Applications). However, it introduces a steep learning curve, requires complex state management (Redux/Zustand), and demands a build step.
 2. *Vanilla HTML, CSS, and JavaScript:* Direct, requires no build tools, and allows absolute control over the DOM.

Outcome: We chose Vanilla HTML/CSS/JS. For a pilot deployment, introducing the overhead of a modern reactive framework is unnecessary. We can achieve all required functionality with standard DOM APIs and **fetch**, prioritizing delivery speed over technical complexity.

ADR 4: Authentication Token (JWT) Handling

- **Context:** UniMentor authenticates Student, Tutor and Admin accounts. Since our Frontend and Backend are decoupled, we need a secure method to transmit and store the JSON Web Token (JWT) to maintain user sessions.
- **Options considered:**
 1. *JWT in the Authorization Header (stored in **localStorage**):* Easy to implement, but **localStorage** is accessible via JavaScript, exposing the tokens to Cross-Site Scripting (XSS) vulnerabilities.
 2. *JWT in HttpOnly Cookies:* The backend sends the token inside a cookie configured with the **HttpOnly** and **Secure** flags.

Outcome: We chose HttpOnly Cookies. It prevents client-side JavaScript from reading the token, which mitigates token theft through XSS (it does not remove XSS itself). The team prioritizes protection against client-side token access. Impact: because the frontend (Vercel) and the backend (Render) are on different sites, the cookie is cross-site. Express must enable CORS with credentials for the Vercel origin, the cookie must use `SameSite=None; Secure`, and state-changing requests need CSRF protection (e.g., a custom header or CSRF token). Browsers that block third-party cookies can break this setup; serving the API under the same site (a Vercel rewrite/proxy or a shared parent domain) is the fallback.

ADR 5: Interactive Calendar System Library

- **Context:** Tutors publish availability and students book appointments. Our frontend stack relies entirely on vanilla JS rather than React or Angular.
- **Options considered:**
 1. *Toast UI Calendar*: A feature-rich alternative, but integration and customization may require more effort during rapid prototyping.
 2. *Custom-built calendar*: Building the grid, date handling, and client-side interaction from scratch. Highest implementation burden.
 3. *FullCalendar (vanilla JS)*: Established, customizable calendar library with extensive documentation and direct JavaScript integration.

Outcome: We chose FullCalendar. Developing a calendar from scratch poses a very high technical risk. FullCalendar fits our vanilla JS stack and handles rendering, selection and navigation of the calendar grid. It does not guarantee conflict-free bookings: that guarantee comes from the PostgreSQL transaction and server-side validation (ADR 2, CU-08), so the calendar is a UI concern only.

ADR 6: Email Notification Provider

- **Context:** Booking confirmations and reminders must reach both students and tutors. The Node.js backend needs an email-delivery solution.
- **Options considered:**
 1. *Nodemailer with a basic SMTP server (e.g., Gmail)*: Easy and inexpensive for development, but a personal Gmail setup has sending limits and operational deliverability concerns.
 2. *SendGrid (API)*: A dedicated transactional-email service with an official Node.js SDK and an API designed for application notifications.

Outcome: We chose SendGrid. A transactional-email service is preferred for confirmations and reminders, with better operational controls (API, delivery logs) and room to scale. Deliverability cannot be guaranteed, and the plan terms and free-tier quotas must be checked before deployment. Impact: the Notification Service wraps the provider behind a single interface, so another provider (or an SMTP fallback) can replace it without touching the business logic. The use cases in Section 2 are written against this choice.

ADR 7: Hosting and Deployment Strategy

- **Context:** The application needs to be accessible over the public internet for the pilot test. We have zero budget for cloud infrastructure.
- **Options considered:**
 1. *AWS EC2 / DigitalOcean Droplets:* Gives absolute control over the Linux server, but requires manual configuration of Nginx, SSL certificates, and CI/CD pipelines.
 2. *Vercel (Frontend) + Render (Backend & Database):* PaaS (Platform as a Service) providers that offer generous free tiers. Vercel automatically deploys frontend changes from GitHub with SSL. Render offers free Node.js hosting and managed PostgreSQL databases.

Outcome: We chose Vercel and Render. This combination provides a fully automated, zero-cost CI/CD pipeline directly from our GitHub repository, allowing the team to focus entirely on software development rather than DevOps infrastructure. Free tiers have limits (service spin-down after inactivity, retention limits on free databases) that must be checked against the providers' current terms before the pilot; if they are not acceptable, the API and database can move to a paid plan or another host without changing the application code.